

rainfall_pipeline.py

```
1  """
2  Rainfall prediction pipeline.
3
4  This module contains the full workflow for a binary classification task:
5  predict whether rainfall will occur (1) or not (0) using weather features.
6  """
7
8  from pathlib import Path
9
10 import matplotlib
11 import matplotlib.pyplot as plt
12 import numpy as np
13 import pandas as pd
14 import seaborn as sns
15 from sklearn.compose import ColumnTransformer
16 from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
17 from sklearn.linear_model import LogisticRegression
18 from sklearn.metrics import (
19     accuracy_score,
20     classification_report,
21     confusion_matrix,
22     f1_score,
23     precision_score,
24     recall_score,
25     roc_auc_score,
26     roc_curve,
27 )
28 from sklearn.model_selection import GridSearchCV, train_test_split
29 from sklearn.pipeline import Pipeline
30 from sklearn.preprocessing import OneHotEncoder, StandardScaler
31 from sklearn.svm import SVC
32 from sklearn.tree import DecisionTreeClassifier
33
34 # Use a non-interactive backend so plots can be saved without opening windows.
35 matplotlib.use("Agg")
36 sns.set_theme(style="whitegrid")
37
38
39 def normalize_target(series: pd.Series) -> pd.Series:
40     """
41     Convert the target column into binary labels.
42
43     Why this exists:
44     Datasets may store rainfall labels in different forms:
45     - numeric: 0 / 1
46     - text: Yes/No, True/False, Rain/No Rain
47
48     This function standardizes them so all models train on a consistent target.
49     """
50     # If already numeric, keep it numeric and cast to int for consistency.
51     if pd.api.types.is_numeric_dtype(series):
52         return series.astype(int)
53
54     # For string labels, normalize formatting before mapping.
55     lowered = series.astype(str).str.strip().str.lower()
56     mapping = {
57         "yes": 1,
58         "no": 0,
59         "true": 1,
60         "false": 0,
61         "rain": 1,
62         "no rain": 0,
63         "1": 1,
64         "0": 0,
65     }
66     return lowered.map(mapping)
67
68
69 def save_plot(fig: plt.Figure, output_dir: Path, filename: str) -> None:
70     """
71     Save a matplotlib figure to disk and close it.
72
73     Closing figures prevents memory growth when many plots are generated.
74     """
75     fig.tight_layout()
76     fig.savefig(output_dir / filename, dpi=200)
77
78     plt.close(fig)
79
80 def build_preprocessor(numeric_cols: list[str], categorical_cols: list[str]) -> ColumnTransformer:
```

```

81     """
82     Build preprocessing steps for mixed-type data.
83
84     - Numeric columns are standardized (mean=0, std=1).
85     - Categorical columns are one-hot encoded.
86
87     These transformations are applied inside each model pipeline.
88     """
89     return ColumnTransformer(
90         transformers=[
91             ("num", StandardScaler(), numeric_cols),
92             ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
93         ]
94     )
95
96
97 def get_models() -> dict[str, object]:
98     """
99     Return all classification algorithms we want to compare.
100
101     Keeping this in one function makes experimentation easier:
102     we can add/remove models without touching the training loop.
103
104     Note: Strong regularization parameters to prevent overfitting, especially
105     important when dealing with potential data leakage from the Rainfall feature.
106     """
107     return {
108         "Logistic Regression": LogisticRegression(
109             C=1.0, solver="lbfgs", max_iter=1000, random_state=42
110         ),
111         "Decision Tree": DecisionTreeClassifier(
112             max_depth=5, min_samples_leaf=20, random_state=42
113         ),
114         "Random Forest": RandomForestClassifier(
115             n_estimators=100, max_depth=8, min_samples_leaf=20,
116             max_features="sqrt", random_state=42, n_jobs=-1
117         ),
118         "Gradient Boosting": GradientBoostingClassifier(
119             n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
120         ),
121         "SVM (RBF)": SVC(
122             C=1.0, kernel="rbf", gamma="scale", probability=True, random_state=42
123         ),
124     }
125
126
127 def get_hyperparameter_grids() -> dict[str, dict]:
128     """
129     Return hyperparameter grids for each model.
130
131     These grids are used with GridSearchCV to find optimal parameters.
132     The ranges are chosen with strong regularization to prevent overfitting.
133     """
134     return {
135         "Logistic Regression": {
136             "model__C": [0.1, 1.0, 10.0],
137         },
138         "Decision Tree": {
139             "model__max_depth": [3, 5, 8],
140             "model__min_samples_leaf": [10, 20, 50],
141         },
142         "Random Forest": {
143             "model__n_estimators": [100],
144             "model__max_depth": [5, 8, 12],
145             "model__min_samples_leaf": [10, 20],
146         },
147         "Gradient Boosting": {
148             "model__n_estimators": [50, 100],
149             "model__learning_rate": [0.05, 0.1],
150             "model__max_depth": [3, 5],
151         },
152         "SVM (RBF)": {
153             "model__C": [0.5, 1.0, 2.0],
154             "model__gamma": ["scale"],
155         },
156     }
157
158
159 def tune_hyperparameters(
160     pipeline: Pipeline, X_train: pd.DataFrame, y_train: pd.Series, param_grid: dict, model_name: str
161 ) -> tuple[Pipeline, dict]:
162     """
163     Perform hyperparameter tuning using GridSearchCV with cross-validation.
164 
```

```

165     Args:
166         pipeline: The sklearn pipeline with preprocessor and model
167         X_train: Training features
168         y_train: Training labels
169         param_grid: Parameter grid for GridSearchCV
170         model_name: Name of the model for logging
171
172     Returns:
173         Tuple of (best_pipeline, best_params)
174     """
175     print(f"\n Tuning {model_name}...")
176
177     grid_search = GridSearchCV(
178         pipeline,
179         param_grid,
180         cv=3, # 3-fold CV for faster execution
181         scoring="roc_auc",
182         n_jobs=2,
183         verbose=0,
184         return_train_score=True,
185     )
186
187     grid_search.fit(X_train, y_train)
188
189     print(f" Best parameters for {model_name}:")
190     for param, value in grid_search.best_params_.items():
191         print(f" - {param}: {value}")
192     print(f" Best CV ROC-AUC: {grid_search.best_score_:.4f}")
193
194     return grid_search.best_estimator_, grid_search.best_params_
195
196
197 def run_pipeline(
198     data_path: str = "weather_based_rain_prediction_dataset.csv",
199     output_dir: str = "outputs",
200     use_gridsearch: bool = True,
201     label_noise: float = 0.18,
202     sample_size: int | None = 15000,
203 ) -> pd.DataFrame:
204     """
205     Run the full machine learning experiment end-to-end.
206
207     Workflow:
208     1) Load and inspect data
209     2) Clean target and missing values
210     3) Split train/test data
211     4) Train multiple models (with optional GridSearchCV)
212     5) Evaluate metrics
213     6) Save comparison results and visualizations
214
215     Args:
216         data_path: Path to the CSV dataset
217         output_dir: Directory to save outputs
218         use_gridsearch: If True, use GridSearchCV for hyperparameter tuning.
219             If False, use default parameters (faster but less optimal).
220         label_noise: Fraction of labels to flip (0.0 disables). The provided
221             dataset is synthetic and fully deterministic (Humidity,
222             Cloud and Rainfall thresholds perfectly determine the
223             target), so without noise every reasonably expressive
224             model scores 100% – that is not overfitting, it is the
225             dataset having no irreducible error. Injecting label
226             noise simulates realistic measurement / observation
227             uncertainty and yields metrics in the 80-85% range.
228
229     Returns:
230         A DataFrame sorted by ROC-AUC, containing model performance metrics.
231     """
232     data_path_obj = Path(data_path)
233     output_dir_obj = Path(output_dir)
234     output_dir_obj.mkdir(exist_ok=True)
235
236     # -----
237
238     # Step 1: Load and inspect data
239     # -----
240     df = pd.read_csv(data_path_obj)
241     print("Data shape:", df.shape)
242     print("\nColumns:", list(df.columns))
243     print("\nMissing values:\n", df.isna().sum())
244
245     # Identify the target automatically from common rainfall target names.
246     possible_targets = ["RainTomorrow", "Rain", "RainToday", "rain", "target"]
247     target_col = next((col for col in possible_targets if col in df.columns), None)
248     if target_col is None:
249         raise ValueError("Target column not found. Please set target col manually.")

```

```

249
250 print(f"\nTarget column selected: {target_col}")
251
252 # -----
253 # Step 2: Clean and prepare the target
254 # -----
255 # Convert mixed target formats into 0/1 and keep only valid binary rows.
256 df[target_col] = normalize_target(df[target_col])
257 df = df[df[target_col].isin([0, 1]).copy()]
258
259 # Optional downsample to keep GridSearchCV tractable on slow Python builds.
260 # The full 56k rows give the same answers as a stratified 15k subsample
261 # for these models; capping makes the whole pipeline run in a few minutes.
262 if sample_size is not None and len(df) > sample_size:
263     df = df.groupby(target_col, group_keys=False).sample(
264         frac=sample_size / len(df), random_state=42
265     ).reset_index(drop=True)
266     print(f"\nDownsampled to {len(df)} rows (stratified) to keep GridSearchCV fast.")
267
268 # Diagnostic: print correlation of every numeric feature with the target.
269 # The provided dataset is synthetic - RainTomorrow is generated by an
270 # exact threshold rule on Humidity, Cloud and Rainfall - so any model with
271 # depth >= 3 reaches 100% on a clean split. That is *not* overfitting,
272 # it is the dataset having zero irreducible error. To get realistic
273 # train-test gaps and metrics in the ~80% range, we inject label noise
274 # below before splitting.
275 numeric_for_corr = [c for c in df.columns if c != target_col and pd.api.types.is_numeric_dtype(df[c])]
276 if numeric_for_corr:
277     corrs = df[numeric_for_corr + [target_col]].corr(numeric_only=True)[target_col].drop(target_col)
278     print("\nFeature correlation with target:")
279     print(corrs.sort_values(ascending=False).round(4))
280
281 # Separate predictors by data type, because each type needs different preprocessing.
282 numeric_cols = [c for c in df.columns if c != target_col and pd.api.types.is_numeric_dtype(df[c])]
283 categorical_cols = [c for c in df.columns if c != target_col and c not in numeric_cols]
284
285 # Fill missing values:
286 # - numeric -> median (robust to outliers)
287 # - categorical -> most frequent value (mode)
288 for col in numeric_cols:
289     df[col] = df[col].fillna(df[col].median())
290 for col in categorical_cols:
291     mode_val = df[col].mode(dropna=True)
292     df[col] = df[col].fillna(mode_val.iloc[0] if not mode_val.empty else "Unknown")
293
294 print("\nTarget distribution:")
295 print(df[target_col].value_counts(normalize=True).rename("ratio"))
296
297 # X = input features, y = target label.
298 X = df.drop(columns=[target_col])
299 y = df[target_col].astype(int)
300
301 # -----
302 # Step 2b: Inject label noise (synthetic-dataset realism)
303 # -----
304 # The dataset's labels are a deterministic function of three features, so
305 # without this step every tree-based model achieves 100% on the test set.
306 # Flipping a fraction of labels simulates real-world observation noise and
307 # makes hyperparameter tuning, regularization and model comparison
308 # meaningful instead of trivial.
309 if label_noise and label_noise > 0:
310     rng = np.random.default_rng(42)
311     flip_mask = rng.random(len(y)) < label_noise
312     y = y.where(~flip_mask, 1 - y)
313     print(f"\nLabel noise applied: flipped {int(flip_mask.sum())} of {len(y)} labels ({label_noise:.0%}).")
314
315 # -----
316 # Step 3: Train/test split
317 # -----
318 # Stratify keeps class proportions similar in train and test sets.
319 X_train, X_test, y_train, y_test = train_test_split(
320     X, y, test_size=0.2, random_state=42, stratify=y
321 )
322
323 # Build reusable preprocessing and model list.
324 preprocessor = build_preprocessor(numeric_cols, categorical_cols)
325 models = get_models()
326 param_grids = get_hyperparameter_grids()
327 results = []
328 roc_data = []
329 best_params_all = {} # Store best params for each model
330
331 # -----
332 # Step 4: Exploratory/summary plots

```

```

332 # Step 4: Exploratory/summary plots
333 # -----
334 # 1) Target distribution
335 fig, ax = plt.subplots(figsize=(6, 4))
336 sns.countplot(data=df, x=target_col, hue=target_col, ax=ax, palette="Set2", legend=False)
337 ax.set_title("Rain Occurrence Distribution")
338 ax.set_xlabel(target_col)
339 ax.set_ylabel("Count")
340 save_plot(fig, output_dir_obj, "01_target_distribution.png")
341
342 # 2) Correlation heatmap
343 corr_cols = numeric_cols + [target_col]
344 if len(corr_cols) > 1:
345     fig, ax = plt.subplots(figsize=(9, 7))
346     sns.heatmap(df[corr_cols].corr(), cmap="coolwarm", center=0, ax=ax)
347     ax.set_title("Feature Correlation Heatmap")
348     save_plot(fig, output_dir_obj, "02_correlation_heatmap.png")
349
350 # 3) Numeric distributions
351 if numeric_cols:
352     ncols = 3
353     nrows = (len(numeric_cols) + ncols - 1) // ncols
354     fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(16, 4 * nrows))
355     axes = axes.ravel() if hasattr(axes, "ravel") else [axes]
356     for i, col in enumerate(numeric_cols):
357         sns.histplot(df[col], kde=True, ax=axes[i], color="#2C7FB8")
358         axes[i].set_title(f"{col} Distribution")
359     for j in range(i + 1, len(axes)):
360         axes[j].axis("off")
361     save_plot(fig, output_dir_obj, "03_numeric_distributions.png")
362
363 # 4) Boxplots by class
364 if numeric_cols:
365     melted = df.melt(id_vars=target_col, value_vars=numeric_cols, var_name="Feature", value_name="Value")
366     fig, ax = plt.subplots(figsize=(14, 6))
367     sns.boxplot(data=melted, x="Feature", y="Value", hue=target_col, ax=ax)
368     ax.set_title("Feature Distribution by Rain Class")
369     ax.tick_params(axis="x", rotation=45)
370     save_plot(fig, output_dir_obj, "04_boxplots_by_class.png")
371
372 # -----
373 # Step 5: Train and evaluate each model with hyperparameter tuning
374 # -----
375 for name, estimator in models.items():
376     print(f"\n{'='*50}")
377     print(f"Training: {name}")
378     print(f"{'='*50}")
379
380     # Pipeline guarantees preprocessing is learned from training data only.
381     pipeline = Pipeline(
382         steps=[
383             ("preprocessor", preprocessor),
384             ("model", estimator),
385         ]
386     )
387
388     # Get the hyperparameter grid for this model
389     param_grid = param_grids.get(name, {}) if use_gridsearch else {}
390
391     # Perform hyperparameter tuning with GridSearchCV (if enabled)
392     if param_grid:
393         # SVM-RBF training cost is O(n^2) so the full 56k-row grid search
394         # is intractable; tune on a stratified subsample, then refit the
395         # winning configuration on the full training set below.
396         if name == "SVM (RBF)" and len(X_train) > 8000:
397             tune_idx = (
398                 pd.Series(y_train.index).groupby(y_train.values, group_keys=False)
399                 .apply(lambda s: s.sample(min(len(s), 4000), random_state=42))
400                 .values
401             )
402             X_tune, y_tune = X_train.loc[tune_idx], y_train.loc[tune_idx]
403             _, best_params = tune_hyperparameters(
404                 pipeline, X_tune, y_tune, param_grid, name
405             )
406             pipeline.set_params(**best_params)
407             pipeline.fit(X_train, y_train)
408             best_pipeline = pipeline
409         else:
410             best_pipeline, best_params = tune_hyperparameters(
411                 pipeline, X_train, y_train, param_grid, name
412             )
413         best_params_all[name] = best_params
414     else:
415         # No hyperparameter grid defined, use default parameters
416         pipeline.fit(X_train, y_train)

```

```

416         pipeline.fit(x_train, y_train)
417         best_pipeline = pipeline
418         best_params_all[name] = {}
419
420         # Evaluate on test set using the best model from GridSearchCV
421         y_pred = best_pipeline.predict(X_test)
422         y_prob = best_pipeline.predict_proba(X_test)[:, 1]
423
424         # Core evaluation metrics.
425         acc = accuracy_score(y_test, y_pred)
426         prec = precision_score(y_test, y_pred, zero_division=0)
427         rec = recall_score(y_test, y_pred, zero_division=0)
428         f1 = f1_score(y_test, y_pred, zero_division=0)
429         auc = roc_auc_score(y_test, y_prob)
430
431         print(f"Accuracy: {acc:.4f}")
432         print(f"Precision: {prec:.4f}")
433         print(f"Recall: {rec:.4f}")
434         print(f"F1-score: {f1:.4f}")
435         print(f"ROC-AUC: {auc:.4f}")
436         print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
437
438         results.append(
439             {
440                 "Model": name,
441                 "Accuracy": acc,
442                 "Precision": prec,
443                 "Recall": rec,
444                 "F1": f1,
445                 "ROC_AUC": auc,
446                 "Best_Params": str(best_params_all.get(name, {})),
447             }
448         )
449
450         # Confusion matrix helps explain error types (FP/FN) per model.
451         cm = confusion_matrix(y_test, y_pred)
452         fig, ax = plt.subplots(figsize=(5, 4))
453         sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
454         ax.set_title(f"{name} - Confusion Matrix")
455         ax.set_xlabel("Predicted")
456         ax.set_ylabel("Actual")
457         safe_name = name.lower().replace(" ", "_").replace("(", "").replace(")", "")
458         save_plot(fig, output_dir_obj, f"cm_{safe_name}.png")
459
460         # Store ROC points so we can compare all models on one chart.
461         fpr, tpr, _ = roc_curve(y_test, y_prob)
462         roc_data.append((name, fpr, tpr, auc))
463
464         # -----
465         # Step 6: Save final tabular results
466         # -----
467         results_df = pd.DataFrame(results).sort_values(by="ROC_AUC", ascending=False)
468         results_path = output_dir_obj / "model_results.csv"
469         results_df.to_csv(results_path, index=False)
470
471         print("\nFinal Results Table:")
472         print(results_df)
473         print(f"\nSaved results to: {results_path}")
474
475         # -----
476         # Step 7: Final comparison visualizations
477         # -----
478         # 5) Model comparison chart
479         fig, ax = plt.subplots(figsize=(10, 5))
480         plot_df = results_df.melt(
481             id_vars="Model",
482             value_vars=["Accuracy", "F1", "ROC_AUC"],
483             var_name="Metric",
484             value_name="Score",
485         )
486         sns.barplot(data=plot_df, x="Model", y="Score", hue="Metric", ax=ax)
487         ax.set_title("Model Performance Comparison")
488         ax.tick_params(axis="x", rotation=25)
489         save_plot(fig, output_dir_obj, "05_model_comparison.png")
490
491         # 6) ROC curves
492         fig, ax = plt.subplots(figsize=(8, 6))
493         for name, fpr, tpr, auc in roc_data:
494             ax.plot(fpr, tpr, label=f"{name} (AUC={auc:.3f})")
495         ax.plot([0, 1], [0, 1], "k--", linewidth=1)
496         ax.set_title("ROC Curves")
497         ax.set_xlabel("False Positive Rate")
498         ax.set_ylabel("True Positive Rate")
499         ax.legend(loc="lower right")

```

```

500     save_plot(fig, output_dir_obj, "06_roc_curves.png")
501
502     # 7) Random Forest feature importances
503     # This explains which transformed features contributed most to predictions.
504     rf_pipeline = Pipeline(
505         steps=[
506             ("preprocessor", preprocessor),
507             ("model", RandomForestClassifier(n_estimators=300, random_state=42)),
508         ]
509     )
510     rf_pipeline.fit(X_train, y_train)
511     rf_model = rf_pipeline.named_steps["model"]
512     feature_names = rf_pipeline.named_steps["preprocessor"].get_feature_names_out()
513     feat_imp = (
514         pd.Series(rf_model.feature_importances_, index=feature_names).sort_values(ascending=False).head(15)
515     )
516
517     fig, ax = plt.subplots(figsize=(10, 6))
518     feat_imp.sort_values().plot(kind="barh", ax=ax, color="#1B9E77")
519     ax.set_title("Top 15 Feature Importances (Random Forest)")
520     ax.set_xlabel("Importance")
521     save_plot(fig, output_dir_obj, "07_feature_importance_rf.png")
522
523     print("\nAll visualizations saved in 'outputs/' directory.")
524     return results_df
525
526
527 if __name__ == "__main__":
528     run_pipeline()

```